Workshop Overview

01

Query Processing Questions

02

Lab: More, more SQL

Query Processing

- Queries can be expensive
  - Different for selection / projection / joins
- Want to make them as efficient as possible!

- Idea:
  - The worst query requires a full scan of the database
  - With different storage & indexing methods, you can reduce the number of records you need to scan

Cost

Depends on:

- **File organisation**
- **Reduction Factor**
  - Estimates % **portion** of the relation that satisfies the given condition

Review of File Organisation

- Heap File
- Sorted File
- Index:
  - Hash Index
  - B-Tree Index

Q1. Effect of index on Selection

Consider a relation R (a,b,c,d,e) containing 5,000,000 records, where each data page of the relation holds 10 records. R is organized as a sorted file with secondary indexes. Assume that R.a is a candidate key for R, with values lying in the range 0 to 4,999,999, and that R is stored in R.a order. For each of the following relational algebra queries, state which of the following three approaches is most likely to be the cheapest:

- Access the sorted file of R directly.
- Use a B+ tree index on attribute R.a.
- Use a hash index on attribute R.a.

Queries:

a. $\sigma_{a < 50000}(R)$
b. $\sigma_{a = 50000}(R)$
c. $\sigma_{a > 50000 \wedge a < 50010}(R)$

Q1. Effect of index on Selection

Consider a relation R (a,b,c,d,e) containing 5,000,000 records, where each data page of the relation holds 10 records. R is organized as a sorted file with secondary indexes. Assume that R.a is a candidate key for R, with values lying in the range 0 to 4,999,999, and that R is stored in R.a order. For each of the following relational algebra queries, state which of the following three approaches is most likely to be the cheapest:

- Access the sorted file of R directly.
- Use a B+ tree index on attribute R.a.
- Use a hash index on attribute R.a.

Queries:

a. $\sigma_{a < 50000}(R)$
b. $\sigma_{a = 50000}(R)$
c. $\sigma_{a > 50000 \wedge a < 50010}(R)$

Range selection → need sorting → where does it start?

Equality selection → do we need sorting?

Q1. Effect of index on Selection

Consider a relation R (a,b,c,d,e) containing 5,000,000 records, where each data page of the relation holds 10 records. R is organized as a sorted file with secondary indexes. Assume that R.a is a candidate key for R, with values lying in the range 0 to 4,999,999, and that R is stored in R.a order. For each of the following relational algebra queries, state which of the following three approaches is most likely to be the cheapest:

- Access the sorted file of R directly.
- Use a B+ tree index on attribute R.a.
- Use a hash index on attribute R.a.

Queries:

a. $\sigma_{a < 50000}(R)$
b. $\sigma_{a = 50000}(R)$
c. $\sigma_{a > 50000 \wedge a < 50010}(R)$

Sorted file of R directly

Hash index

B+ tree index

Primary Conjunct

- Selection condition → ***predicate***
- Primary conjunct:
  - Matching index & predicate pairs / combinations
  - An index matches (a combination of) predicates that involve only attributes in a **prefix** of the search key

- Index on <a, b, c> will match predicates on:
  - <a, b, c>, <a, b>, <a>
  - E.g. primary conjuncts can be (a=3 ∧ b>5)
  - Cannot be used to answer b=3

a	b	c	
1	3	1	
2	1	1	b=3
2	3	1	
3	2	1	
3	6	1	
3	6	4	
6	3	1	

Q2. Matching Index

Consider the following schema for the Sailors relation:

Sailors (sid INT, sname VARCHAR(50), rating INT, age DOUBLE)

For each of the following indexes, list whether the index matches the given selection conditions and briefly explain why.

- A B+ tree index on the search key (Sailors.sid)
  a. $\sigma_{\text{Sailors.sid} < 50{,}000}(\text{Sailors})$
  b. $\sigma_{\text{Sailors.sid} = 50{,}000}(\text{Sailors})$
- A hash index on the search key (Sailors.sid)
  c. $\sigma_{\text{Sailors.sid} < 50{,}000}(\text{Sailors})$
  d. $\sigma_{\text{Sailors.sid} = 50{,}000}(\text{Sailors})$
- A B+ tree index on the search key (Sailors.rating, Sailors.age)
  e. $\sigma_{\text{Sailors.rating} < 8 \wedge \text{Sailors.age} = 21}(\text{Sailors})$
  f. $\sigma_{\text{Sailors.rating} = 8}(\text{Sailors})$
  g. $\sigma_{\text{Sailors.age} = 21}(\text{Sailors})$

Q2. (a)

- A B+ tree index on the search key (Sailors.sid)
  a. $\sigma_{\text{Sailors.sid} < 50{,}000}(\text{Sailors})$
  b. $\sigma_{\text{Sailors.sid} = 50{,}000}(\text{Sailors})$

Match:
Primary conjunct: *Sailors.sid < 50000*

Match:
Primary conjunct: *Sailors.sid = 50000*

Q2. (b)

- A hash index on the search key (Sailors.sid)
  - c. $\sigma_{\text{Sailors.sid} < 50{,}000}(\text{Sailors})$
  - d. $\sigma_{\text{Sailors.sid} = 50{,}000}(\text{Sailors})$

NO Match:
Range queries don’t match w/ hash index

Match:
Primary conjunct: *Sailors.sid = 50000*

Q2. (b)

- A B+ tree index on the search key (Sailors.rating, Sailors.age)
  - e. $\sigma_{\text{Sailors.rating} < 8 \wedge \text{Sailors.age} = 21}(\text{Sailors})$
  - f. $\sigma_{\text{Sailors.rating} = 8}(\text{Sailors})$
  - g. $\sigma_{\text{Sailors.age} = 21}(\text{Sailors})$

Match:
Primary conjunct:
- Sailors.rating < 8

Match:
Primary conjunct:
- Sailors.rating < 8; AND
- Sailors.rating < 8 ∧ Sailors.age = 21

NO Match:
- The index on *(Sailors.rating, Sailors.age)* is primarily sorted on *Sailors.rating*
- *So t*he entire relation would need to be searched to find those sailors with a particular *Sailors.age* value

Q3. Cost of Joins

5. **Joins (between relations R and S, R = outer, S = inner) Cost**
   a. NLJ
      i. Tuple-oriented NLJ

         Cost = NPages(R) + NTuples(R) * NPages(S)

      ii. Page-oriented NLJ

         Cost = NPages(R) + NPages (R) * NPages(S)

      iii. Block-oriented NJL (for block_size B)

         Cost = NPages(R) + ceil(NPages (R)/(B-2)) * NPages(S)

   b. Hash Join

      Cost = 3*(NPages(R) + NPages(S))

   c. Sort-Merge Join

      $Cost_{SMJ}$ = NPages(R) + NPages(S) +

      2* NPages(R)* num_passes(R) +

      2* NPages(S)* num_passes(S)

B = # buffer pages

Q3. Cost of Joins

Consider the join $R \bowtie_{R.a = S.b} S$, given the following information about the relations to be joined:

- Relation R contains 10,000 tuples and has 10 tuples/page.
- Relation S contains 2,000 tuples and also has 10 tuples/page.
- Attribute b of relation S is the primary key for S.
- Both relations are stored as simple heap files.
- Neither relation has any indexes built on it.
- 52 buffer pages are available.

The cost metric is the number of page I/Os unless otherwise noted and the cost of writing out the result should be uniformly ignored.

Which relation should be the outer relation?

Q3. Cost of Joins

Consider the join $R \bowtie_{R.a = S.b} S$, given the following information about the relations to be joined:

- Relation R contains 10,000 tuples and has 10 tuples/page.
- Relation S contains 2,000 tuples and also has 10 tuples/page.
- Attribute b of relation S is the primary key for S.
- Both relations are stored as simple heap files.
- Neither relation has any indexes built on it.
- 52 buffer pages are available.

The cost metric is the number of page I/Os unless otherwise noted and the cost of writing out the result should be uniformly ignored.

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

Q3(a):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

Consider the join R $\bowtie_{R.a = S.b}$ S, given the following information about the relations to be joined:

- Relation R contains 10,000 tuples and has 10 tuples/page.
- Relation S contains 2,000 tuples and also has 10 tuples/page.
- Attribute b of relation S is the primary key for S.
- Both relations are stored as simple heap files.
- Neither relation has any indexes built on it.
- 52 buffer pages are available.

The cost metric is the number of page I/Os unless otherwise noted and the cost of writing out the result should be uniformly ignored.

a. What is the cost of joining R and S using the **page-oriented Simple Nested Loops** algorithm? What is the minimum number of buffer pages (in memory) required in order for this cost to remain unchanged?

Q3(a):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

a. What is the cost of joining R and S using the **page-oriented Simple Nested Loops** algorithm? What is the minimum number of buffer pages (in memory) required in order for this cost to remain unchanged?

- Total cost = (# of pages in outer) + (# of pages in outer × # of pages in inner)
  - = N + (N × M) = 200 + (200 × 1000) = 200,200

- One input buffer to page through each relation (2 in total)
- One output buffer to store the output.
- → (1*2)+1 = 3 buffer pages are required

Q3(b):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

Consider the join R $\bowtie_{R.a = S.b}$ S, given the following information about the relations to be joined:

- Relation R contains 10,000 tuples and has 10 tuples/page.
- Relation S contains 2,000 tuples and also has 10 tuples/page.
- Attribute b of relation S is the primary key for S.
- Both relations are stored as simple heap files.
- Neither relation has any indexes built on it.
- 52 buffer pages are available.

The cost metric is the number of page I/Os unless otherwise noted and the cost of writing out the result should be uniformly ignored.

b. What is the cost of joining R and S using the **Block Nested Loops** algorithm? What is the minimum number of buffer pages required in order for this cost to remain unchanged?

Q3(b):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

b. What is the cost of joining R and S using the **Block Nested Loops** algorithm? What is the minimum number of buffer pages required in order for this cost to remain unchanged?

$$\#\text{ of blocks} = \text{ceil}\left(\frac{\#\text{ of pages in outer}}{B-2}\right) = \text{ceil}\left(\frac{200}{50}\right) = 4$$

- Total cost = (# of pages in outer) + (# of blocks × # of pages in inner)
  - = 200 + (4 × 1000) = 4200

- If fewer buffers available, the cost will increase as the # of blocks will vary.
- The minimum number of buffer pages is 52 for this cost.

Q3(c):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

Consider the join R $\bowtie_{R.a = S.b}$ S, given the following information about the relations to be joined:

- Relation R contains 10,000 tuples and has 10 tuples/page.
- Relation S contains 2,000 tuples and also has 10 tuples/page.
- Attribute b of relation S is the primary key for S.
- Both relations are stored as simple heap files.
- Neither relation has any indexes built on it.
- 52 buffer pages are available.

The cost metric is the number of page I/Os unless otherwise noted and the cost of writing out the result should be uniformly ignored.

c. What is the cost of joining R and S using the **Sort-Merge Join** algorithm? Assume that the external merge sort process can be completed in 2 passes.

Q3(c):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

c. What is the cost of joining R and S using the **Sort-Merge Join** algorithm? Assume that the external merge sort process can be completed in 2 passes.

- Cost of sorting R = 2 × # of passes × # of pages of R
  - = 2 × 2 × 1000 = 4000
- Cost of sorting S = 2 × 2 × 200 = 800
- Cost of merging R and S = # of pages read of R + # of pages read of S
  - = 1000 + 200 = 1200

- Total cost = Cost of sorting R + Cost of sorting S + Cost of merging R and S
  - = 4000 + 800 + 1200 = 6000

Q3(d):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

Consider the join R $\bowtie_{R.a = S.b}$ S, given the following information about the relations to be joined:

- Relation R contains 10,000 tuples and has 10 tuples/page.
- Relation S contains 2,000 tuples and also has 10 tuples/page.
- Attribute b of relation S is the primary key for S.
- Both relations are stored as simple heap files.
- Neither relation has any indexes built on it.
- 52 buffer pages are available.

The cost metric is the number of page I/Os unless otherwise noted and the cost of writing out the result should be uniformly ignored.

d. What is the cost of joining R and S using the **Hash Join** algorithm?

Q3(d):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

d. What is the cost of joining R and S using the **Hash Join** algorithm?

- In hash join, each relation is partitioned and then the join is performed by “matching” elements from corresponding partitions.

- Total cost = 3(M + N)
  - = 3(1000 + 200) = 3600

Q3(e):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

Consider the join R $\bowtie_{R.a = S.b}$ S, given the following information about the relations to be joined:

- Relation R contains 10,000 tuples and has 10 tuples/page.
- Relation S contains 2,000 tuples and also has 10 tuples/page.
- Attribute b of relation S is the primary key for S.
- Both relations are stored as simple heap files.
- Neither relation has any indexes built on it.
- 52 buffer pages are available.

The cost metric is the number of page I/Os unless otherwise noted and the cost of writing out the result should be uniformly ignored.

e. What would the lowest possible I/O cost be for joining R and S using any join algorithm, and how much buffer space would be needed to achieve this cost? Explain briefly.

Q3(e):

- # pages in R: M = 10,000 / 10 = 1000
- # pages in S: N = 2,000 / 10 = 200
- # buffer pages available: B = 52

e. What would the lowest possible I/O cost be for joining R and S using any join algorithm, and how much buffer space would be needed to achieve this cost? Explain briefly.

- The optimal cost would be achieved if each relation was read only once
  - Store the entire smaller relation in memory, reading in the larger relation page by page
  - For each tuple in the larger relation we search the smaller relation (which exists entirely in memory) for matching tuples.
  - The buffer pool would have to hold the entire smaller relation, one page for reading in the larger relation and one page to serve as an output buffer.

- Total cost = M + N = 1200
- The minimum number of buffer pages for this cost is min{M, N} + 1 + 1 = 202.